

**UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE**

**DEPARTAMENTO DE CIENCIAS DE LA
COMPUTACIÓN**

CARRERA: INGENIERÍA DE SOFTWARE

NRC: 30724

ASIGNATURA: Análisis y Diseño de Software

TEMA: Reporte de Errores - Sistema Fi-Citas

Nombres (Equipo de Pruebas):

- Altamirano, Xavier
- Cordero, Martin
- Ayo, Alisson

DOCENTE: Ing. Jenny Ruiz

FECHA: 14 de Julio del 2026

Índice

1. Introducción	2
2. Registro Detallado de Incidencias	2
3. Estado Actual de Aseguramiento de Calidad	5

Nota de Alcance del Proyecto

Este documento se enfoca exclusivamente en el control de calidad para el agendamiento de citas y gestión de usuarios. Conforme a las directrices del proyecto y el modelo Scrum, el sistema **no maneja expedientes ni historias clínicas**.

1. Introducción

Durante las fases de desarrollo (Sprints) y ejecución de pruebas del sistema web **Fi-Citas**, bajo el marco de trabajo **Scrum**, se implementaron y validaron 109 casos de prueba automatizados. A lo largo de este ciclo, el equipo identificó múltiples fallos, cuellos de botella y vulnerabilidades de seguridad. Todas las incidencias documentadas en este informe fueron gestionadas en el Product/Sprint Backlog, priorizadas y corregidas para alcanzar el 100 % de aprobación en la fase actual de control de calidad.

Este documento presenta el reporte oficial y detallado de las incidencias críticas detectadas y sus respectivas acciones correctivas.

2. Registro Detallado de Incidencias

INC-001: Error Crítico en el Frontend por Carencia de Datos

Nivel de Severidad: Crítico

Estado: Resuelto

Descripción del Fallo:

La aplicación cliente (React) presentaba una interrupción total del servicio, mostrando una pantalla en blanco y registrando la excepción `TypeError: Cannot read properties of undefined (reading 'map')`. Este comportamiento ocurría cuando el servidor experimentaba latencia o devolvía arreglos vacíos en los módulos de pacientes o doctores.

Fase de Detección:

Pruebas de integración en los componentes `Dashboard` y `Appointments`.

Acción Correctiva Aplicada:

Se implementó el manejo de *Zero States* (estados vacíos) en el renderizado de la interfaz. Actualmente, si las estructuras de datos son nulas o están vacías, el sistema carga un mensaje de retroalimentación al usuario (ej. "No hay citas pendientes"), evitando la ruptura del árbol de componentes de React.

INC-002: Incumplimiento de Estándares de Accesibilidad (A11y) en Formularios**Nivel de Severidad:** Medio**Estado:** Resuelto**Descripción del Fallo:**

Los formularios correspondientes a la creación de citas y pacientes no cumplían con los estándares de accesibilidad del W3C. Los lectores de pantalla utilizados por personas con discapacidad visual eran incapaces de asociar semánticamente las etiquetas descriptivas con los campos de entrada de texto.

Fase de Detección:

Ejecución de pruebas automatizadas con `React Testing Library`, la cual reportó fallos al invocar el método `getByLabelText` durante la simulación de interacción de usuario.

Acción Correctiva Aplicada:

Se refactorizó el código HTML/JSX de los formularios para asignar atributos `id` únicos a cada elemento `<input>`, vinculándolos de manera estricta a la propiedad `htmlFor` de sus respectivas etiquetas `<label>`.

INC-003: Vulnerabilidad de Seguridad por Ataque de Fuerza Bruta en Autenticación**Nivel de Severidad:** Crítico (Riesgo de Seguridad)**Estado:** Resuelto**Descripción del Fallo:**

El módulo de inicio de sesión no limitaba la cantidad de intentos fallidos, permitiendo la ejecución de ataques de fuerza bruta para adivinar credenciales y exponiendo al servidor a posibles sobrecargas.

Fase de Detección:

Diseño y ejecución de pruebas de Sistema para el requerimiento de seguridad REQ001.

Acción Correctiva Aplicada:

Se implementó un mecanismo de bloqueo temporal (*Lockout*) a nivel de Backend. Tras registrar tres intentos fallidos consecutivos, el servidor retorna un código HTTP 403 con la propiedad `locked: true`. El Frontend procesa esta respuesta, despliega un temporizador de restricción en pantalla y deshabilita la interacción del usuario hasta que expire el tiempo de penalización.

INC-004: Fallo Lógico en Backend por Solapamiento de Citas**Nivel de Severidad:** Alto**Estado:** Resuelto**Descripción del Fallo:**

El algoritmo de agendamiento carecía de validaciones temporales restrictivas, lo que permitía registrar dos o más citas con el mismo especialista en la misma franja horaria, generando conflictos logísticos irreconciliables.

Fase de Detección:

Ejecución de la suite de pruebas unitarias en el controlador `appointments.test.js` (Escenario: *should prevent overlapping appointments*).

Acción Correctiva Aplicada:

Se incorporó una restricción de validación matemática en el controlador del Backend. El sistema ahora establece un margen de validación de **2 horas**; si se detecta un registro aprobado dentro de esa ventana de tiempo para el mismo doctor, la solicitud es rechazada con un Error HTTP 400.

INC-005: Caída del Servicio por Dependencia de la API de WhatsApp**Nivel de Severidad:** Alto**Estado:** Resuelto**Descripción del Fallo:**

La operación de agendamiento presentaba un alto acoplamiento con la API externa de mensajería (Fonnte). En escenarios donde el servicio externo presentaba caídas o la variable de entorno `FONNTE_TOKEN` no estaba configurada, el *endpoint* generaba una excepción no controlada (Error HTTP 500), bloqueando la inserción de la cita en la base de datos de manera indefinida.

Fase de Detección:

Pruebas de integración del flujo de agendamiento en el archivo `02-integracion.test.js`.

Acción Correctiva Aplicada:

Se desacopló el servicio de notificaciones externas. El envío del mensaje se ejecuta ahora de manera asíncrona mediante un bloque `try/catch`. En caso de falla en la API externa, el sistema registra una advertencia en consola (*Warning: [WhatsApp] FONNTE_TOKEN no configurado | mensaje no enviado*), pero garantiza que la transacción principal de guardado de la cita se complete exitosamente.

3. Estado Actual de Aseguramiento de Calidad

A la fecha de emisión de este reporte, **todas las incidencias documentadas han sido resueltas**. Asimismo, se han integrado estas correcciones a la suite de pruebas de regresión automáticas del proyecto.

Esto garantiza que, dentro del flujo de trabajo continuo y la metodología Scrum, si algún componente futuro interfiere con estas soluciones durante los Sprints siguientes, el sistema de integración (`npm run test`) alertará de forma inmediata la anomalía, previniendo el despliegue de errores conocidos hacia el entorno de producción.